

Lottery Search Benchmark

ค้นสลาก 6 หลักจำนวน **10 ล้านใบ** ด้วย pattern ที่มี wildcard เช่น * * * * 2 3 แล้ว
แจกให้ผู้ใช้พร้อมกันหลายคนโดยห้ามซ้ำ เอกสารนี้เทียบ 4 วิธีทำ index ด้วยตัวเลขที่วัดจริง
บน Kubernetes ที่บังคับ resource limit ทุกตัวเลขมีไฟล์หลักฐานกำกับ

วันที่วัด	ข้อมูล	API POD	POSTGRES POD
2026-09-05	10,000,000 ใบ	500m · 128Mi	2000m · 1Gi
POSTGRESQL	เครื่องมือ	SCENARIO	
17.11	k6 v1.2.3	7 ชุด	

00 สรุปผล

สมมติฐาน 3 ข้อผ่านหมด ด้วย Strategy C

H1 · LATENCY ผ่าน p95 47 ms เป้าคือ p95 ต่ำกว่า 100 ms ทุกคลาส pattern ที่ 100 VUs ทำได้ 6,068 req/s error 0	H2 · NO DUPLICATE ผ่าน 0 ใบซ้ำ จอง 342,412 ใบ ภายได้ 200 VUs ที่ยิง pattern เดียวกัน ไม่มีตัวใบไหนไปถึงสองคน	H3 · FASTER THAN LIKE ผ่าน ~9,500x pattern แบบระบุครบ 6 หลักจาก 829 ms เหลือ 0.087 ms และ index เล็กกว่า Strategy A ครึ่งหนึ่ง
STABILITY ผ่าน 26 / 128 MiB soak 10 นาที 942k requests ไม่ restart ไม่ OOMKilled		

สิ่งที่แนะนำให้ใช้ คือ PostgreSQL 17 ตัวเดียว กับ **Strategy C** สำหรับการค้นหา และใช้ **สุ่มเลขจริงใน pattern แล้ว point lookup** คู่กับ `FOR UPDATE SKIP LOCKED` สำหรับการจอง ไม่ต้องมี Redis ไม่ต้องมี distributed lock

01 โจทย์และกฎสำคัญ

พื้นที่ค้นหาจริงคือ 1 ล้าน ไม่ใช่ 10 ล้าน

สลาก 6 หลักมีค่าที่เป็นไปได้แค่ $10^6 = 1,000,000$ เลข เมื่อมีสลาก 10 ล้านใบ แปลว่า เลขหนึ่งเลขมีสลากเฉลี่ย 10 ใบ ดังนั้นการค้นหา pattern จึงไม่จำเป็นต้องไล่ดูสลากทั้ง 10 ล้านใบ แคหาว่า *เลขไหนบ้างที่เข้า pattern* จากตารางเล็ก 1 ล้านแถวก่อน แล้วค่อย join เข้าตารางสลาก นี่คือการคิดเบื้องหลัง Strategy B และ C

อีกเรื่องที่ต้องเข้าใจก่อนอ่านตัวเลข คือขนาดผลลัพธ์ต่างกันเป็นล้านเท่าตามจำนวน wildcard เราจึงแบ่ง pattern เป็น 4 คลาสแล้ววัดแยกกัน

1 2 3 4 5 6

w0 — ระบุครบ ไม่มี wildcard เข้าเงื่อนไข 1 เลข รว 10 ใบ นี่คือการที่ยากที่สุดสำหรับ index ที่ไม่ดี

1 2 3 * * *

w3 — ล็อกสามหลักหน้า เข้าเงื่อนไข 1,000 เลข รว 10,000 ใบ

* * * * 2 3

w4 — ลงท้ายด้วย 23 เข้าเงื่อนไข 10,000 เลข รว 100,000 ใบ

* * 3 * * *

w5 — ล็อกหลักกลางหลักเดียว เข้าเงื่อนไข 100,000 เลข รว 1,000,000 ใบ

ทำไม B-TREE ธรรมดาไม่พอ

index ปกติบนคอลัมน์ `number` เรียงจากซ้ายไปขวา จึงใช้ได้เฉพาะตอนที่รู้หลักหน้า เช่น

1 2 3 * * *

แต่ถ้า wildcard อยู่หน้าอย่าง * * * * 2 3 index จะช่วยอะไรไม่ได้เลย นี่คือการหัวใจของโจทย์

02 4 กลยุทธ์ INDEX

แต่ละแบบทำอะไร และแลกอะไรไป

ทั้ง 4 แบบใช้ schema เดียวกัน ต่างกันแค่ index และวิธีเขียน query สลับด้วย env var `STRATEGY` ได้ โดยไม่ต้อง migrate ใหม่ ทำให้เทียบกันบนข้อมูลชุดเดียวกันได้จริง

กลยุทธ์	
O Baseline ตัวเปรียบเทียบ	<code>lpad(number::text,6,'0')</code>
A Expression index index ต่อหลัก	สร้าง index แยกทีละหลักบนตาราง <code>tickets</code> แล้วหวังให้ plan
B Dimension table ตาราง 1 ล้านแถว	หาเลขที่เข้า pattern จากตาราง <code>numbers(n, d1..d6)</code> ก่อน แล้ว
C B + prefix range แนะนำ	เหมือน B แต่แปลงหลักหน้าที่ล็อกไว้เป็นช่วง <code>n BETWEEN</code> บน primary key และถ้าระบุครบ 6 หลักก็ยิง
ขนาด index วัดด้วย <code>pg_relation_size</code> ที่ 10M rows ส่วน heap ของ <code>tickets</code> เองอยู่ที่ 438 MB	

ประเด็นสำคัญของ Strategy C คือมันเปลี่ยนงานจาก "ไล่หาทีละหลัก" เป็น "อ่านช่วงเดียวบน primary key" ซึ่ง Postgres ทำได้เร็วมากและเป็น index-only scan ด้วย

```
-- pattern 123*** ภายใต้ Strategy C
number IN (SELECT n FROM numbers WHERE n BETWEEN 123000 AND 123999)

-- pattern 123456 ภายใต้ Strategy C ข้ามตาราง numbers ไปเลย
number = 123456
```

03 ผล EXPLAIN

วัดทีละ query ยังไม่มีโหลด

ก่อนจะยิงโหลด ต้องรู้ก่อนว่า query ใดๆ ใช้เวลาเท่าไร ตัวเลขนี้มาจาก `EXPLAIN (ANALYZE, BUFFERS)` บน PostgreSQL 17.11 ที่ 10M rows โดยขอผลแค่ `LIMIT 20` หน่วยเป็นมิลลิวินาที (cache อุ่นแล้ว)

PATTERN	WILDCARD	0 BASELINE	A	B	C
123456	0	829	130	5.1	0.087
123***	3	2.5	21.6	1.7	0.148
12*4**	3 กระจาย	1.5	21.6	1.3	0.158
****23	4	0.17	0.11	0.22	0.132
3*	5	0.05	0.05	0.33	0.206
*****	6	0.04	0.02	0.02	0.024

จุดที่คนอ่านผิดบ่อย

baseline ดู "เร็ว" ในแถวล่างๆ ไม่ใช่เพราะมันดี แต่เพราะมี LIMIT 20 pattern ที่กว้างมากจะเจอ 20 ใบแรกตั้งแต่ต้นตาราง เลยหยุดสแกนเร็ว เคสที่วัดฝีมือจริงคือแถวบนสุด 1 2 3 4 5 6 ที่มีของตรงเงื่อนไขแค่ ~10 ใบใน 10 ล้านใบ ทำให้ต้องอ่านทั้งตารางกว่าจะครบ 20

04 S1 - LATENCY

ไต่จาก 10 ถึง 100 VUs ยิง /v1/search

รันสักรอบ ข้อมูลชุดเดียวกัน pod เดียวกัน เปลี่ยนแค่ STRATEGY แต่ละรอบสุม pattern จาก 4 คลาส แล้วแยกสถิติตามคลาส หน่วยเป็นมิลลิวินาที

กลยุทธ์	P95 W0	P95 W3	P95 W4	P95 W5	P95 รวม	สูงสุด	REQ/S	ERROR
C numbers + prefix range	46.8	46.9	46.5	46.7	46.7	71	6,068	0 %
B numbers table	176.0	114.1	103.3	103.6	108.3	196	975	0 %
A expression index ต่อหลัก	5,663	4,503	4,503	4,392	4,705	6,514	25	0 %
0 baseline LIKE	1,320	1,049	1,018	1,053	1,051	22,197	128	94.3 %

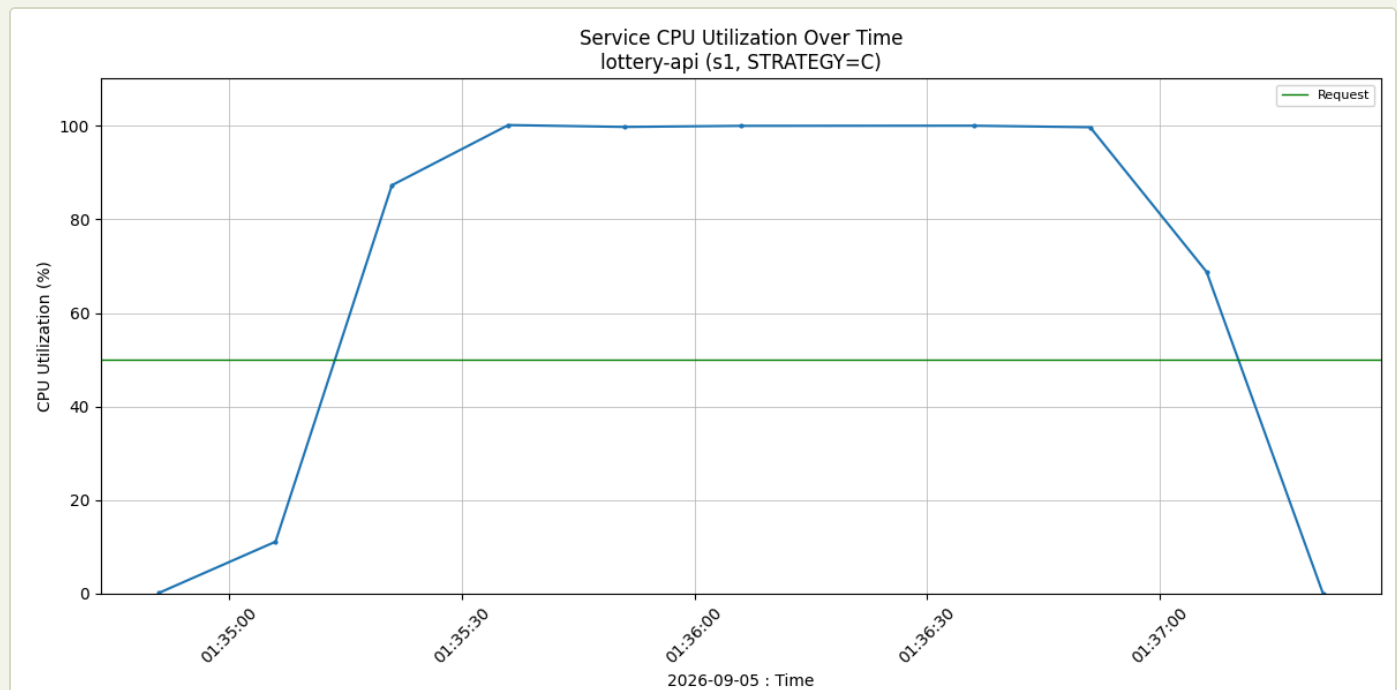
p95 อ่านว่า 95 % ของ request เร็วกว่าค่านี้ ใช้แทนค่าเฉลี่ยเพราะค่าเฉลี่ยซ่อนหางยาวที่ผู้ใช้เจอจริง

คอขวดอยู่ตรงไหน

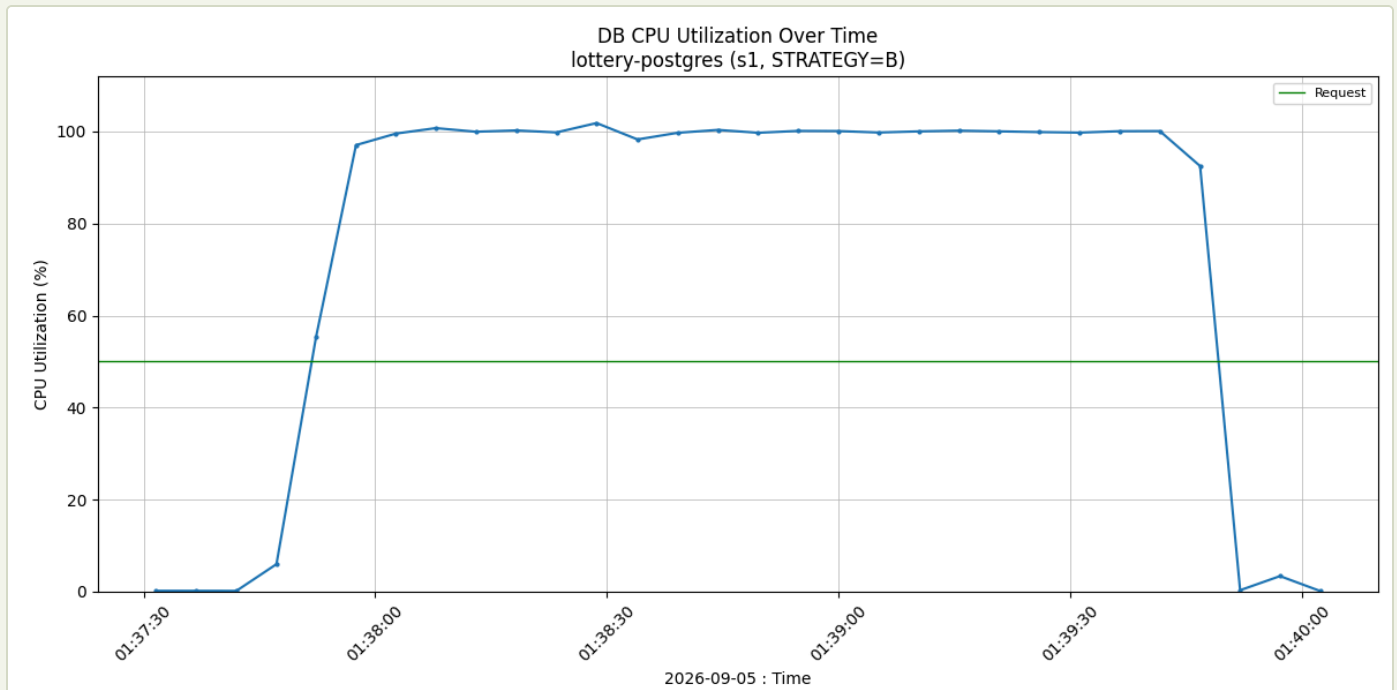
ตารางข้างล่างสำคัญพอๆ กับตาราง latency เพราะมันบอกว่า *ทรัพยากรตัวไหนหมดก่อน* ซึ่งเป็นตัวกำหนดว่าจะขยายระบบต่ออย่างไร

กลยุทธ์	API CPU	API MEM	POSTGRES CPU	POSTGRES WORKING SET	POSTGRES เหลือว่าง
C	100 % ขนเพดาน	17 MiB	85 %	976 MiB	48 MiB
B	24 %	16 MiB	102 % ขนเพดาน	1,007 MiB	17 MiB
A	2 %	12 MiB	102 % ขนเพดาน	999 MiB	25 MiB
O	1 %	9 MiB	101 % ขนเพดาน	789 MiB	235 MiB

C เป็นแบบเดียวที่ย้ายคอขวดออกจาก database ได้ พอ Postgres ไม่ใช่ตัวที่เต็มแล้ว การเพิ่ม replica ของ API จะทำให้ throughput ขึ้นต่อได้ทันที ส่วน B กับ A ต่อให้เพิ่ม API อีกก็ตัวก็ไม่ช่วย เพราะ Postgres เต็มไปแล้ว



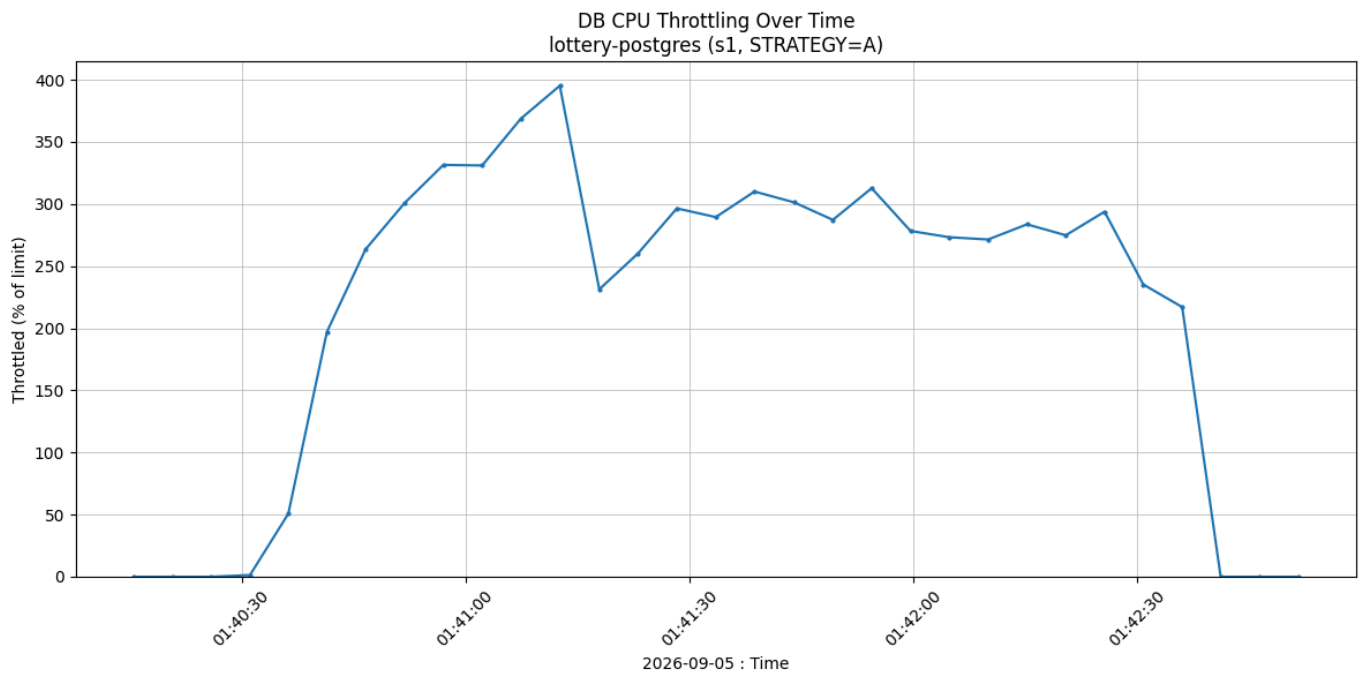
Strategy C — API pod ขนเพดาน เส้นนิ่งที่ 100 % ของ 500m ตลอดช่วงยิงโหลด เส้นเขียวคือค่า request ที่ตั้งไว้ (250m) นี่คือรูปของระบบที่ "ขยายต่อได้"



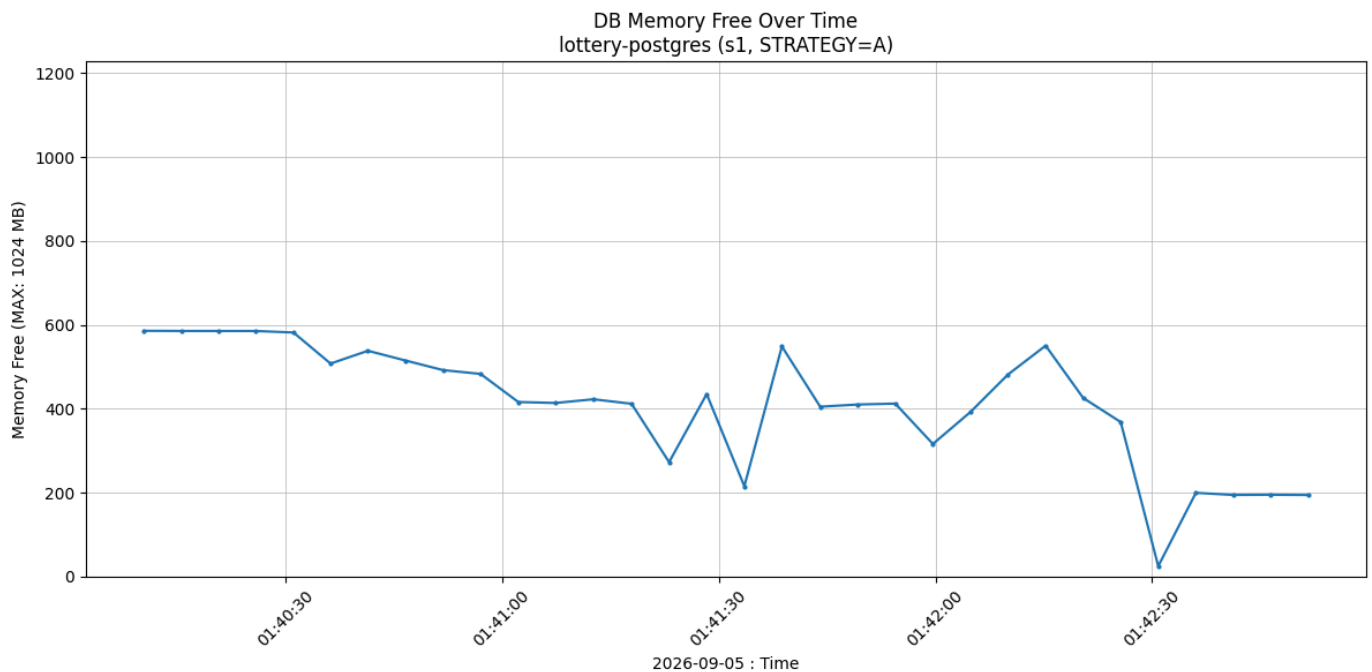
Strategy B – Postgres ขนเพดานแทน ขณะที่ API ใช้แค่ 24 % งานหนักไปกองที่ BitmapAnd ของ digit index บนตาราง numbers

Strategy A เจ็บกว่าที่คิด

A ไม่ได้แค่ช้า แต่ทำให้ Postgres โดน *throttle* คือถูก cgroup ตัดโควตา CPU ทั้ง กราฟซ้ายอ่านว่าในหนึ่งวินาที cgroup ถูกปฏิเสธเวลา CPU ไปมากถึง 395 % ของ limit ที่มี นั่นคือกลไกที่ทำให้ p95 พุ่งไป 4,705 ms ส่วนกราฟขวาคือ memory ที่เหลือ ซึ่งลงไปแตะ 25 MiB จาก 1 GiB



โดนตัดโควตา CPU สูงสุด 395 % ยิ่งตัวเลขสูง แปลว่า process ยิ่งถูกบังคับให้หยุดรอบ่อย



เหลือ 25 MiB จาก 1 GiB ทุก strategy ที่มี index ต่างวิ่งขีด limit 1 GiB เพราะ hot set ของตาราง 10 ล้านแถวพร้อม index ขนาด 405 MB มีขนาดพอๆ กับ pod

log จริงของ Strategy C

นี่คือส่วนสรุปจาก `docs/evidence/s1-C/10-k6.log` เครื่องหมาย ✓ หน้าเมตริกแปลว่า threshold ที่ตั้งไว้ผ่านทั้งหมด ในที่นี้คือ p95 ต่ำกว่า 100 ms และ p99 ต่ำกว่า 250 ms

```
● DOCS/EVIDENCE/S1-C/10-K6.LOG

✓ status 200

checks.....: 100.00% ✓ 728412      ✗ 0
data_received.....: 548 MB   4.6 MB/s
data_sent.....: 120 MB   995 kB/s
http_req_blocked.....: avg=740ns  min=0s      med=1µs    max=630µs  p(90)
http_req_connecting.....: avg=21ns   min=0s      med=0s     max=375µs  p(90)
✓ http_req_duration.....: avg=8.78ms min=321µs   med=4.97ms max=70.55ms p(90)
✓ { class:w0 }.....: avg=8.87ms min=321µs   med=5.03ms max=70.55ms p(90)
✓ { class:w3 }.....: avg=8.89ms min=345µs   med=5.06ms max=69.66ms p(90)
✓ { class:w4 }.....: avg=8.61ms min=346µs   med=4.87ms max=70.37ms p(90)
✓ { class:w5 }.....: avg=8.75ms min=351µs   med=4.92ms max=68.86ms p(90)
  { expected_response:true }...: avg=8.78ms min=321µs   med=4.97ms max=70.55ms p(90)
✓ http_req_failed.....: 0.00%    ✓ 0      ✗ 728412
http_req_receiving.....: avg=9.68µs min=3µs     med=8µs    max=1.71ms  p(90)
http_req_sending.....: avg=3.01µs min=1µs     med=2µs    max=781µs   p(90)
http_req_tls_handshaking.....: avg=0s     min=0s     med=0s     max=0s      p(90)
http_req_waiting.....: avg=8.77ms min=311µs   med=4.96ms max=70.54ms p(90)
http_reqs.....: 728412  6068.075842/s
iteration_duration.....: avg=8.81ms min=340.54µs med=5ms    max=70.58ms p(90)
iterations.....: 728412  6068.075842/s

running (2m00.0s), 000/100 VUs, 728412 complete and 0 interrupted iterations
```

log จริงของ baseline ที่ล่ม

รอบ baseline ไม่ได้แค่ช้าลง แต่ล่ม pool ขนาด 10 connection เต็มไปด้วย query ที่กินเวลาระดับวินาที ทำให้ `/readyz` ตอบไม่ทัน Kubernetes จึงถอด pod ออกจาก Service แล้ว request ที่เหลือก็ถูกตัดกลางคัน สังเกตว่า checks ผ่านแค่ 5.66 % และ 15,588 จาก 15,592 ครั้งที่พังเป็น error แบบ `E0F`

```
● DOCS/EVIDENCE/S1-0/10-K6.LOG

checks.....: 5.66% ✓ 936      ✗ 15592
data_received.....: 712 kB   5.5 kB/s
data_sent.....: 2.7 MB   21 kB/s
http_req_blocked.....: avg=146.91µs min=1µs     med=105µs   max=8.37ms  p(
http_req_connecting.....: avg=126.51µs min=0s     med=91µs    max=5.49ms  p(
```



```

x http_req_duration.....: avg=405.97ms min=0s      med=350µs    max=22.19s p(
x { class:w0 }.....: avg=571.09ms min=0s      med=351µs    max=22.19s p(
x { class:w3 }.....: avg=376.21ms min=0s      med=352µs    max=16.18s p(
x { class:w4 }.....: avg=299.37ms min=0s      med=340µs    max=14.41s p(
x { class:w5 }.....: avg=376.16ms min=0s      med=357µs    max=16.1s  p(
  { expected_response:true }...: avg=6.54s    min=446µs    med=6.4s     max=22.19s p(
x http_req_failed.....: 94.33% ✓ 15592      x 936
http_req_receiving.....: avg=3.46µs   min=0s       med=0s       max=2.12ms p(
http_req_sending.....: avg=17.54µs  min=0s       med=11µs     max=3.75ms p(
http_req_tls_handshaking.....: avg=0s       min=0s       med=0s       max=0s     p(
http_req_waiting.....: avg=405.95ms min=0s       med=332µs    max=22.19s p(
http_reqs.....: 16528  127.779244/s
iteration_duration.....: avg=406.21ms min=240.66µs med=596.66µs max=22.19s p(
iterations.....: 16528  127.779244/s

```

running (2m09.3s), 000/100 VUs, 16528 complete and 0 interrupted iterations

บทเรียนที่เอาไปใช้ได้ทันที

เพราะตอนนี้เราจึงแยก `/healthz` ที่เช็คแค่ว่า process ยังอยู่ ออกจาก `/readyz` ที่ ping database **liveness probe** ห้ามแตะ **database** ไม่งั้นตอน database แน่น Kubernetes จะฆ่า pod ที่งัดเข้าเต็ม ส่วน readiness ควรแตะ เพื่อให้ pod ที่รับงานไม่ไหวถูกถอดออกจาก load balancer ชั่วคราว

05 S2 – CONTENTION

200 คนแย่ง pattern เดียวกัน 60 วินาที

นี่คือข้อที่โจททย์ให้ความสำคัญที่สุด ถ้าผู้ใช้ 200 คนค้น * * * * 2 3 พร้อมกัน ต้องไม่มีใครได้ตั๋วใบเดียวกัน กลไกที่ใช้คือคำสั่งเดียวจบใน transaction เดียว

```

WITH picked AS (
  SELECT id FROM tickets
  WHERE status = 0 AND <pattern>
  LIMIT $2
  FOR UPDATE SKIP LOCKED          -- ← หัวใจ: ใครล็อกแถวไหนไว้ คนถัดไปข้ามไปหยิบใบอื่น
), upd AS (
  UPDATE tickets SET status = 1, reserved_by = $1 FROM picked WHERE tickets.id = picked.id
  RETURNING id, number
), ledger AS (
  INSERT INTO allocations (ticket_id, user_id) SELECT id, $1 FROM upd

```

```
)  
SELECT id, number FROM upd;
```

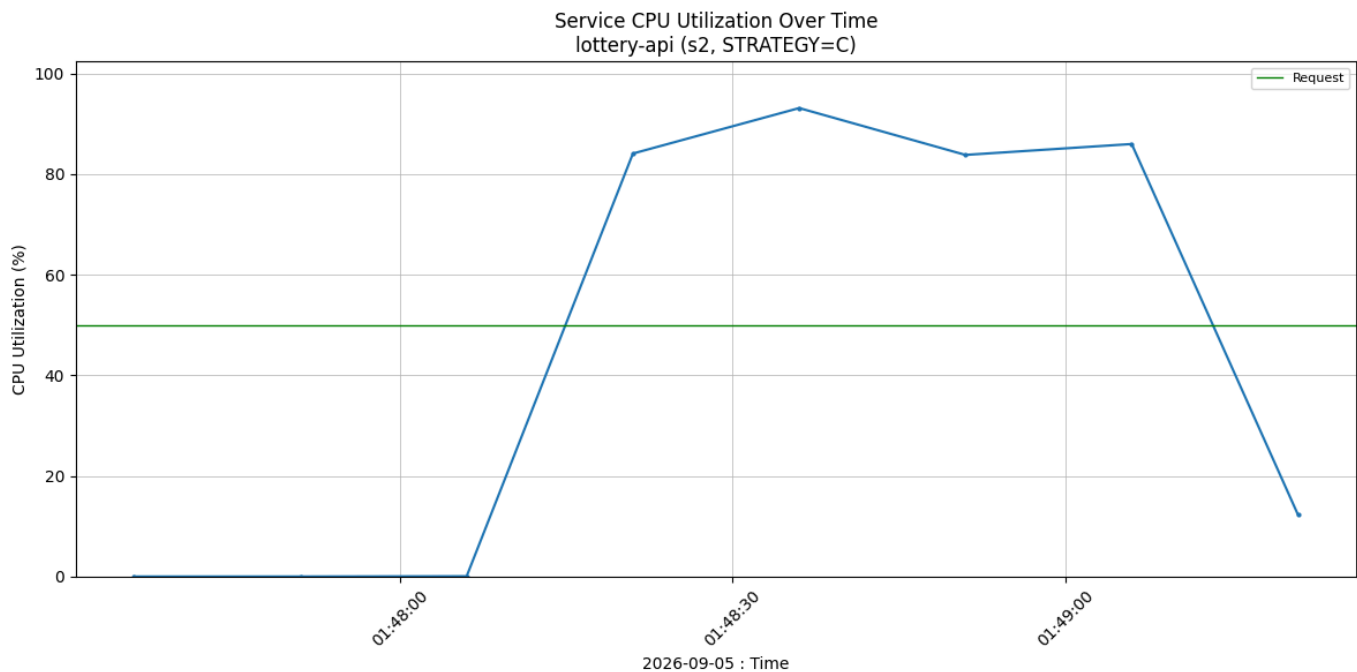
SKIP LOCKED ทำให้ transaction ที่มาทีหลังข้ามแถวที่ถูกล็อกอยู่ แทนที่จะต่อคิวรอ ผลคือไม่มี lock contention latency จึงไม่ฟุ้ง และไม่มีโอกาสเกิด deadlock เพราะไม่มีใครรอใคร

ตัวชี้วัด	PATTERN ****23 ของมีราว 100K ใบ	PATTERN **3*** ของมีราว 1M ใบ
จองได้ใน 60 วินาที	95,630 1,540/s	246,782 4,110/s
HTTP error	0	0
แถวใน ledger = ตัวที่ถูกจอง = จำนวน id ที่ไม่ซ้ำ	95,630	246,782
ตัวที่ถูกจ่ายซ้ำ	0	0
ledger ไม่ตรงกับสถานะตัว	0	0
latency p95 / สูงสุด	510 ms / 2.4 s	75 ms / 760 ms
CPU API / Postgres	80 % / 100 %	93 % / 88 %

H2 ผ่านทั้งสองรอบ รวม 342,412 ใบ ไม่มีใบไหนไปถึงสองคน ตรวจสอบด้วย SQL บรรทัดเดียวที่ต้องได้ 0 แถว

● DOCS/EVIDENCE/S2B/30-VERIFY.TXT

```
ledger rows      : 246782  
reserved tickets : 246782  
distinct ticket  : 246782  
DUPLICATES (H2, must be 0): 0  
ledger vs status mismatch : 0
```



ตอนแย่งจองกัน คอขวดยังอยู่ที่ API ไม่ใช่ database API pod วิ่งขึ้นไปถึง 93 % ขณะที่ Postgres อยู่ที่ 88 % แปลว่า SKIP LOCKED ไม่ได้สร้างการรอคิวที่ database ซึ่งเป็นผลลัพธ์ที่ต้องการ ถ้าใช้ FOR UPDATE เจ๊ๆ กราฟนี้จะกลับด้านทันที

ทำไมสองคอลัมน์ latency ต่างกันมาก

ไม่ใช่เพราะ concurrency พัง แต่เพราะ * * * * 2 3 ถูกจองไป 96 % ภายใน 60 วินาที ช่วงท้ายของรอบทดสอบ การสุ่มเลขจึงไปเจอแต่เลขที่ถูกจองไปแล้ว ต้องตกไปใช้วิธีสแกน pattern บน index ที่เต็มไปด้วยแถวตายแล้ว ส่วน * * 3 * * * มีของเป็นล้านใบ จองยังงี้ก็ไม่หมดใน 60 วินาที ตัวเลข 75 ms จึงเป็นภาพของสถานะปกติ ส่วน 510 ms คือภาพตอนของใกล้หมด ซึ่งในระบบจริงควรจัดการที่ระดับ product เช่นขึ้นข้อความว่าเลขชุดนี้ใกล้หมดแล้ว

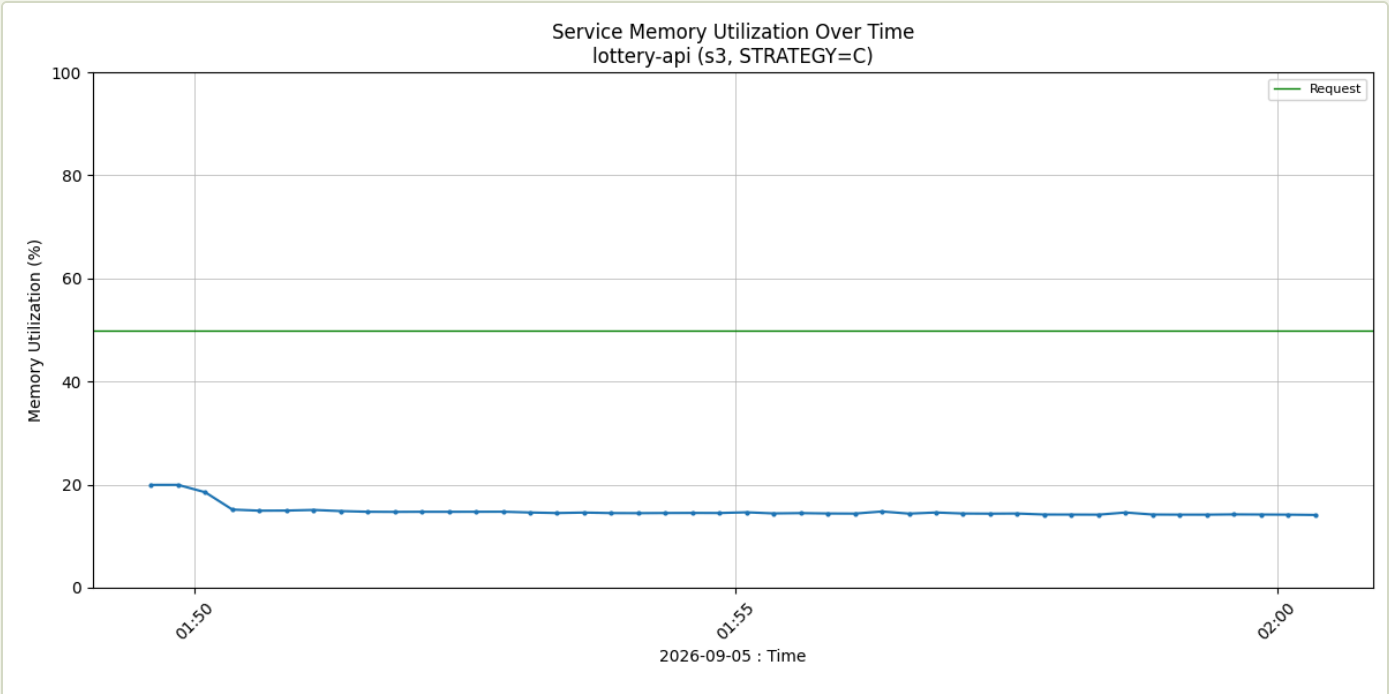
กับดักที่เจอตอนทำ

ตอนแรกใช้ ORDER BY id LIMIT n FOR UPDATE SKIP LOCKED ตามสเปก ปรากฏว่าเกิดอาการ "หวัรร้อน" คือแถวที่ถูกจองไปแล้วยังกองอยู่หน้าสุดของลำดับ id ทุกครั้งที่จองใหม่จึงต้องเดินผ่านของเก่าทั้งหมด วัดได้ว่าหลังจองไปแค่ 713 ใบ ต้องเดินผ่าน 68,453 แถว ใช้เวลา 145 ms ต่อครั้ง และแย่งเรื่อยๆ ทางแก้คือสุ่มเลขจริงที่อยู่ใน pattern เช่น * * * * 2 3 → 481723 แล้วยิง point lookup ถ้าพลาดค่อยลองใหม่สูงสุด 3 ครั้ง ต้นทุนจึงคงที่ไม่ว่าจะจองไปแล้วกี่ใบ โดย SKIP LOCKED ยังเป็นตัวรับประกันความถูกต้องเหมือนเดิม

50 VUs ต่อเนื่อง 10 นาที ทั้ง search จอง และคืน

เป้าหมายของรอบนี้ไม่ใช่ความเร็ว แต่คือดูว่ารันยาวแล้ว memory รั่วหรือโดน OOMKilled ไหม โดยเฉพาะเมื่อ API pod ถูกจำกัดไว้แค่ 128Mi

ตัวชี้วัด	ผล	อ่านว่าอย่างไร
จำนวน request	942,015	1,568 req/s ตลอด 10 นาที
HTTP error	0	ไม่มี request ไหนพังเลย
latency p95 / สูงสุด	104 ms / 6.8 s	ไม่ผ่านเป้า 100 ms เพราะรอบนี้เขียนหนัก
memory ของ API pod สูงสุด	26 MiB	จาก limit 128 MiB เหลือที่ว่างอีกเยอะ
CPU ของ API pod สูงสุด	58 %	จาก 500m
restart / OOMKilled	0 / ไม่มี	ผ่านเกณฑ์ความเสถียร
แถวใน ledger	1,556,674	จากตัวที่ไม่ซ้ำกัน 1,336,481 ใบ
ถูกจองซ้ำหลังคืนของ	220,193	เป็นพฤติกรรมที่ถูกต้อง ไม่ใช่บั๊ก
แถวที่สถานะไม่สอดคล้อง / ledger ลอย	0 / 0	ข้อมูลไม่เพี้ยนเลยตลอด 10 นาที



memory ของ API pod ตลอด 10 นาที เริ่มที่ราว 20 % แล้วลงมาแบนอยู่ราว 15 % ของ limit ไม่ได้ขึ้นเรื่อยๆ ถ้ามี memory leak เส้นนี้จะค่อยๆ ได้ขึ้นจนชนเพดานแล้วโดน OOMKilled คำนี้นี้ได้มาจากการตั้ง `GOMEMLIMIT=100MiB` และ `GOMAXPROCS=1` ให้ Go runtime รู้ว่ามี cgroup limit อยู่

ตอนแรกสคริปต์ตรวจรายงานว่ามีตัวซ้ำ 129,739 ไบ ซึ่งดูเหมือน H2 พัง แต่ที่จริงเป็น **query** ตรวจที่เขียนผิดเอง เพราะ S3 เป็น scenario เดียวที่เรียก `/v1/release` ตัวที่ถูกคืนแล้วถูกจองใหม่เป็นสิ่งที่ถูกต้อง และ ledger แบบ append-only ก็บันทึกทั้งสองครั้งตามจริง ตอนนี้ S3 จึงใช้ชุดตรวจคนละแบบ คือดูความสอดคล้องของสถานะและ ledger ลอย ซึ่งได้ 0 ทั้งคู่ ส่วน H2 พิสูจน์ด้วย S2 ที่ไม่มีการคืนของเลย

● DOCS/EVIDENCE/S3/10-K6.LOG

✓ status 200

```
checks.....: 100.00% ✓ 942015      ✗ 0
data_received.....: 387 MB   644 kB/s
data_sent.....: 168 MB   280 kB/s
http_req_blocked.....: avg=1.1µs   min=0s      med=1µs      max=2.63ms p(90)
http_req_connecting.....: avg=40ns    min=0s      med=0s       max=869µs p(90)
✗ http_req_duration.....: avg=31.82ms min=1.75ms  med=12.77ms  max=6.79s p(90)
  { expected_response:true }...: avg=31.82ms min=1.75ms  med=12.77ms  max=6.79s p(90)
✓ http_req_failed.....: 0.00%     ✓ 0          ✗ 942015
http_req_receiving.....: avg=14.59µs min=4µs     med=10µs     max=5.44ms p(90)
http_req_sending.....: avg=4.95µs  min=1µs     med=4µs      max=6.13ms p(90)
http_req_tls_handshaking.....: avg=0s      min=0s      med=0s       max=0s     p(90)
http_req_waiting.....: avg=31.8ms  min=1.74ms  med=12.76ms  max=6.79s p(90)
http_reqs.....: 942015  1567.852283/s
iteration_duration.....: avg=95.61ms min=18.18ms med=57.78ms  max=7s     p(90)
iterations.....: 314005  522.617428/s
```

running (10m00.8s), 00/50 VUs, 314005 complete and 0 interrupted iterations

07 บทเรียนสำหรับ JUNIOR

สิ่งที่ผิดพลาดระหว่างทาง มีค่ากว่าตัวเลขที่สวยงาม

ระหว่างทำ benchmark ชุดนี้มีความผิดพลาดสามอย่างที่ทุกอย่างดูเหมือนจะทำงานได้ปกติ ถ้าไม่ตรวจให้ละเอียดตัวเลขที่ผิดไปเลย

สิ่งที่เกิดขึ้น

ยิงผิดเป้า
k6 ไปที่
port ของ

ได้ 4 ล้าน request ที่ 68,000/s พร้อม summary หน้าตาปกติ แต่ที่จริงคือ conn

สิ่งที่เกิดขึ้น	
dev server ที่ปิดอยู่	
เวลาเป็น จำนวนเต็ม date +%s คืนค่าเป็น วินาทีเต็ม	ช่วงเวลาจริง 5.2 วินาทีถูกหารด้วย 5 ทำให้ทุกกราฟที่คิดจากอัตราสูงเกินจริง 4 % เห็นเป็น CPU 104 % ที่
ตรวจผิด เงื่อนไข ใช้ query เดียวกับทุก scenario	soak test รายงานตัวซ้ำ 129,739 ใบ เหมือนระบบพัง ทั้งที่เป็นการจอง

สามข้อที่เอาไปใช้กับงานอื่นได้เลย

- **ต้องมีตัวเปรียบเทียบเสมอ** ถ้าไม่ได้วัด baseline ที่ไม่มี index เลย ก็ไม่มีทางรู้ว่า 47 ms ที่ได้นั้นดีจริงหรือแค่ดูดี
- **วัดว่าจะอะไรเดิมก่อน** ไม่ใช่แค่วัดว่าเร็วแค่ไหน latency บอกว่าตอนนี้เป็นอย่างไง แต่กราฟ CPU บอกว่าขยายต่อทางไหนได้
- **ผลที่ไม่ผ่านคือข้อมูล ไม่ใช่ความล้มเหลว** การที่ Strategy A ทำให้ Postgres โดนตัดโควตา CPU ถึง 395 % คือหลักฐานชั้นดีที่พิสูจน์ว่าทำไมถึงไม่ควรเลือกมัน

08 คำศัพท์

ศัพท์ที่โผล่ในรายงานนี้

p95	เวลาที่ 95 % ของ request เร็วกว่าค่านี้ ใช้แทนค่าเฉลี่ยเพราะค่าเฉลี่ยจะซ่อน request ช้าๆ ส่วนน้อยที่ผู้ใช้รู้สึกได้จริง
VU	virtual user คือผู้ใช้จำลองหนึ่งคนใน k6 ที่ยิง request วนไปเรื่อยๆ 200 VUs คือจำลอง 200 คนพร้อมกัน
SKIP LOCKED	ตัวเลือกของ SELECT ... FOR UPDATE ที่สั่งว่าถ้าแถวไหนถูก transaction อื่นล็อกอยู่ ให้ข้ามไปเลยแทนที่จะรอ เหมาะกับการแจกของจากคิว

BitmapAnd	วิธีที่ Postgres รวมผลจากหลาย index เข้าด้วยกัน จะคุ้มก็ต่อเมื่อแต่ละ index คัดของทั้งได้เยอะ ซึ่ง index ของเลขหลักเดียวคัดได้แค่ 1 ใน 10 จึงมักไม่คุ้ม
working set	memory ที่ใช้จริงโดยไม่นับ page cache ที่ระบบเรียกคืนได้ ถ้าดูแค่ตัวเลข memory ดิบ Postgres จะดูเหมือนใช้เกือบเต็ม limit ตลอดเวลา เพราะมันเอา memory วางไปทำ cache
CPU throttling	เมื่อ container ใช้ CPU เกินโควตาที่ตั้งไว้ kernel จะบังคับให้หยุดรอจนหมดรอบเวลา ยิ่งโดนบ่อย latency ยิ่งพุ่ง
liveness / readiness	liveness ตอบว่า process ยังมีชีวิตอยู่ไหม ถ้าตอบไม่ผ่าน Kubernetes จะฆ่าทิ้งแล้วสร้างใหม่ ส่วน readiness ตอบว่าตอนนี้พร้อมรับงานไหม ถ้าไม่ผ่านจะแค่ถูกถอดออกจาก load balancer ชั่วคราว
exit code 99	ค่าที่ k6 คืนเมื่อรันจบครบแต่มี threshold ที่ตั้งไว้ไม่ผ่าน ไม่ได้แปลว่าการทดสอบพัง

09 ข้อจำกัด

สิ่งที่ตัวเลขชุดนี้ยังไม่ได้พิสูจน์

- ทุกอย่างรันบน docker-desktop node เดียว ทั้ง API, Postgres และ k6 จึงแย่ง CPU กันเอง ตัวเลข throughput สัมบูรณ์จะต่างออกไปถ้าแยกเครื่อง แต่การเปรียบเทียบระหว่าง **strategy** ซึ่งเป็นแกนของข้อสรุป ไม่ได้รับผลกระทบ
- กราฟ disk free วัดทั้ง volume ของ VM ไม่ใช่ device เฉพาะของ database เพราะ PVC ถูก back ด้วย host path จึงใช้ดูคร่าวๆ เท่านั้น
- กราฟ CPU และ memory ของ API มาจาก metrics-server ที่อัปเดตทุกราว 15 วินาที ส่วนของ Postgres อ่านจาก cgroup ทุก 5 วินาที ความละเอียดจึงไม่เท่ากันโดยตั้งใจ
- ยังไม่ได้ทดสอบกรณีที่ของถูกจองไปแล้ว 80 % ตั้งแต่ต้น ซึ่งจะกระทบ selectivity ของ partial index
- PoC นี้ไม่มี authentication, payment, ระบบงวด, replication และ caching layer ตามที่ระบุไว้ว่าอยู่นอกขอบเขต

ตัวเลขทุกตัวในรายงานนี้มาจากไฟล์ใน `docs/evidence/` แต่ละ scenario มี 19 ไฟล์ ประกอบด้วย log ของ k6 แบบเต็ม, summary เป็น JSON และ HTML, กราฟ 9 รูป, ข้อมูลดิบ CSV ที่ใช้วาดกราฟ และผลตรวจ SQL รันซ้ำได้ด้วย `make capture-all` และวาดกราฟใหม่จากข้อมูลเดิมได้ด้วย `make charts`

`DIR=docs/evidence/s1-C`